

Exercise 7 — Error Diagnosis Challenge

Exercise Description

Select an error scenario, use the "Tracing Error Messages and Stack Traces" prompts to analyse it, and document the error description, root cause, suggested solution, and learning points. (No runnable fixed code required.)

I analysed the two **Python** scenarios that ship with starter code (`debug-errors-001/python`), plus concise diagnoses of the remaining scenarios.

Scenario 1 — Off-by-One `IndexError` (Python, `stock_manager.py`)

```
for i in range(len(items) + 1):    # + 1 here
    print(f"Item {i+1}: {items[i]['name']} - Quantity: {items[i]['quantity']}")
```

`IndexError: list index out of range`

- **Error description:** the program tries to read a list element at an index that doesn't exist. With 3 items, valid indices are 0-2, but the loop reaches index 3.
- **Root cause:** `range(len(items) + 1)` iterates `0..len(items)` inclusive. On the final iteration `i == len(items)`, and `items[i]` is out of bounds. Classic off-by-one — the `+ 1` is wrong.
- **Suggested solution:** iterate `range(len(items))`, or better, iterate the list directly and use `enumerate` for the 1-based label: `python for i, item in enumerate(items, start=1): print(f"Item {i}: {item['name']} - Quantity: {item['quantity']}")`
- **Learning points:** prefer iterating the collection (`for item in items`) over index arithmetic; when you need a counter, use `enumerate`; off-by-one errors cluster around `len()`, `range`, and `<=` vs `<`.

Scenario 2 — `MemoryError` (Python, `image_processor.py`)

`MemoryError: Unable to allocate 4.8 GiB for array with shape (5000, 5000, 64) and data type float64`

- **Error description:** the process asked the OS for more contiguous memory than is available for a single array.
- **Root cause:** two compounding issues — (a) each "image" is materialised as a giant `5000×5000×64 float64` array (~15 GB per array as built via nested comprehensions), and (b) `process_images` appends every processed array to `all_image_data`, holding them **all** in memory at once instead of processing and releasing one at a time.
- **Suggested solution:** process images in a streaming fashion (handle one, write/aggregate the result, drop the reference before the next); use a smaller dtype (`float32` / `uint8`) where precision allows; avoid Python nested list comprehensions for

pixel data — let the imaging/ `numpy` library allocate directly; if all results really are needed, use a generator or write intermediate results to disk.

- **Learning points:** watch total memory = per-item size × items held simultaneously; choose the smallest sufficient dtype; prefer streaming/generators over accumulating large collections; profile with `tracemalloc/memory_profiler`.

Scenario 3 — `StackOverflowError` (Java, factorial)

- **Description/root cause:** unbounded recursion — `calculateFactorial(num)` calls itself with `num - 1` but has **no base case**, so it recurses past 0 into negatives forever until the call stack is exhausted.
- **Solution:** add a base case `if (num <= 1) return 1;`. Guard against negative input.
- **Learning points:** every recursion needs a base case that is provably reached; consider an iterative form for large `n` (also avoids stack limits and `int` overflow).

Scenario 4 — `NullPointerException` (Java, ShoppingCart)

- **Description/root cause:** a `null Product` is added to the cart, then `calculateTotal()` calls `product.getPrice()` on it.
- **Solution:** reject nulls at the boundary (`addItem` throws on `null`), and/or skip/guard nulls in the loop; use `Optional` or validation.
- **Learning points:** validate inputs at the edge rather than deep in the logic; fail fast with a clear message; NPEs are usually a missing invariant ("items are never null").

Scenario 5 — `TypeError: reading '0' of undefined` (JavaScript, userList)

- **Description/root cause:** `renderUserList` loops `for (i=0; i<5; i++)` assuming 5 users, but only 3 exist, so `users[i]` is `undefined` and `.name` throws.
- **Solution:** iterate `users.length` (`for (let i=0; i<users.length; i++)` or `users.forEach()`), not a hard-coded 5.
- **Learning points:** never hard-code collection sizes; drive loops from the data's actual length.

Scenario 6 — `TypeError: ...map is not a function` (JavaScript, taskManager)

- **Description/root cause:** `addTask` redeclares `let tasks = {...}` (a single object) inside the function, shadowing the module-level `tasks` array; `displayTasks` then calls `.map` on an object.
 - **Solution:** don't shadow — assign to the existing array (`tasks.push(newTask)`), removing the inner `let`.
 - **Learning points:** beware accidental shadowing with `let / const`; keep a single source of truth for shared state; a linter (no-shadow) catches this.
-

Reflection Questions

1. **How did the AI explanation compare to online docs?** Faster and context-specific — it pointed at the exact line and the *why*, whereas docs explain the error class generically.
 2. **What would have been hard to diagnose manually?** The `MemoryError` — the trace shows *where* allocation failed but not that the real problem is accumulation across the loop; that needs reasoning about total memory over time.
 3. **How would you improve error messages?** Validate inputs early with descriptive messages (e.g. `addItem` rejecting null with "product must not be null"); log sizes/counts before big allocations.
 4. **Did the AI help with underlying concepts?** Yes — off-by-one patterns, base-case reasoning, boundary validation, and memory-over-time were all reusable concepts, not just one-line fixes.
-

Submission for Exercise 7 — Error Diagnosis Challenge

1. **Error description / what it means:** provided per scenario above (e.g. Scenario 1: reading a non-existent list index; Scenario 2: single-allocation request exceeds available memory).
2. **Root cause identification:** Scenario 1 — `range(len(items)+1)` off-by-one; Scenario 2 — oversized `float64` arrays **plus** holding all results in memory; plus root causes for the Java (missing base case; null in list) and JS (hard-coded length; `let` shadowing) scenarios.
3. **Suggested solution:** per scenario — `enumerate / range(len(...))`; stream + smaller dtype; add base case; reject nulls at boundary; loop by `length`; remove the shadowing `let`.
4. **Learning points to prevent recurrence:** iterate collections not indices; size = per-item × items-held; every recursion needs a reachable base case; validate at the edges; never hard-code sizes; avoid variable shadowing (use a linter).
5. **Reflection answers:** provided above.